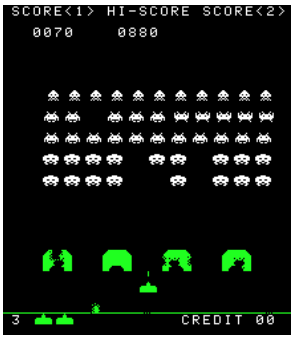
Assignment 3 – Standard Template Library [Part 2]

Goal

The goal of this assignment is to put into practice the use of the Standard Template Library (STL) and lambda expressions. Students will implement a classic **Space Invaders** game incorporating an object pooling mechanism based on **STL algorithms** and event handling using **Lambda Expressions**. Through this assignment, students will gain practical experience in the design and implementation of an object pool and event handling.

The resulting game should be a simplified version of the original, in which the player must destroy a flotilla or squadron of enemy ships. Once the entire squadron has been destroyed, a new flotilla arrives, and the game advances to the next level.



Description

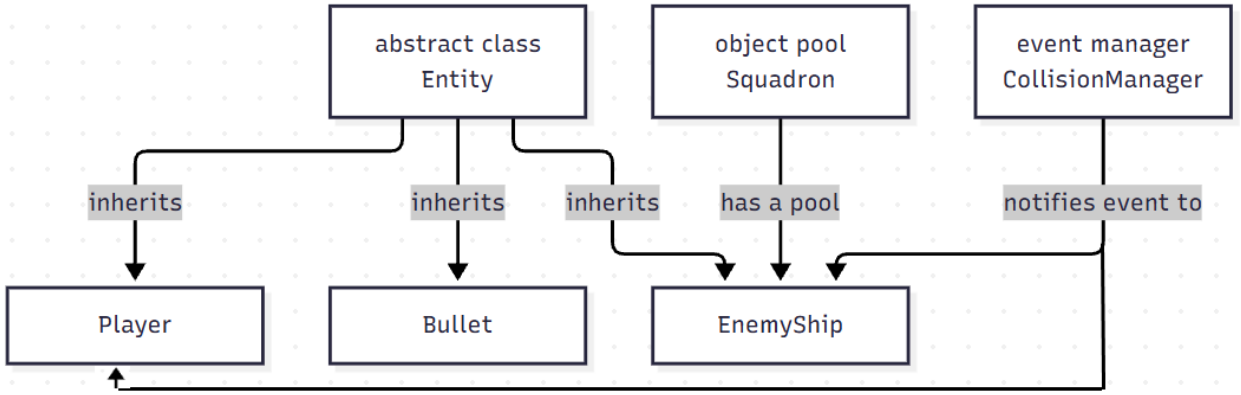
This is an open assignment that can be implemented in different ways, extending and building upon the provided framework. However, the game itself must remain very simple. All enemy ships can be identical and move at a constant speed. All squadrons can contain the same number of ships. No special ship types or movement patterns are required. Also, other elements such as obstacles and UI can be omitted.

A new squadron must appear only when the last ship in the current squadron has been destroyed. The game should follow an infinite-game model. Only when the player loses all their lives the game state must be restarted. Enemy ships must be stored in a container and activated or deactivated using an object pooling mechanism based on STL containers and algorithms. The objective is to reduce runtime overhead, improve memory management, and minimize the overhead associated with dynamic memory allocation and deallocation.

To handle destruction events, whether caused by player bullets or enemy bullets, an event-handling mechanism using lambda expressions must be implemented. When the player destroys an enemy ship, that ship must be deactivated in the pool and must no longer be drawn. When an enemy bullet hits the player, the player's number of lives must be reduced. Deactivated enemy ships can be activated again once the entire squadron has been destroyed. No other event-handling mechanisms are required.

You may use the provided demo as a reference, but you are free to develop your own implementation.

One possible class organization, which could be an ideal starting point, is shown below:



What to do:

Student must submit the complete **src** folder, with all the cc, hh and cpp files.

Submission Guidelines

Submit all the files as a zip folder on Moodle with the following naming conventions:
cs225fa26_{username}_3.2.zip